# CS6886W Systems Engineering for Deep Learning, Assignment 1

Exploring VGG6 on CIFAR-10 with different configurations


| | |
|---|---|
| Name | Anmol Tiwari |
| Roll number | CS25M503 |
| **GitHub repository** | **https://github.com/cs25m503-sudo/vgg6-cifar10** |
| W&B project | https://forge.coreweave.com/wandb/cs25m503-iit-madras/vgg6-cifar10 |
| Best result | **88.80%** validation top-1, **88.40%** test top-1 (ReLU, BN, Nesterov-SGD, lr 0.1, batch 128, 150 epochs) |


**Setup used for every experiment.** The VGG6 architecture follows the course template: `[64, 64, M, 128, 128, M]`. Each Conv3×3 is followed by optional BatchNorm and the activation, with 2×2 max-pooling, then global average pooling and `Linear(128, 10)`, for 262,218 parameters and the template's weight initialization. The official 50,000 training images are split, with seed 42, into **45,000 for training and 5,000 for validation**. Configurations are selected on validation accuracy, and the 10,000 official test images are reported once for each selected checkpoint. Unless stated otherwise, the defaults are: ReLU, BatchNorm, SGD with momentum 0.9, lr 0.05 with cosine annealing to 0, weight decay 5e-4, batch size 128, cross-entropy loss, and seed 42. All runs used one NVIDIA RTX 4090 Laptop GPU with PyTorch 2.14.1 (CUDA 12.6) and were logged to W&B: 60 runs in total, consisting of 1 baseline and 4 sweeps.

## Question 1. Training Baseline

### (a) Data preparation: normalization and augmentation

The normalization statistics are computed directly from the 50,000 CIFAR-10 training images, as the per-channel mean and standard deviation over all pixels in [0, 1] ( `src/data.py: compute_mean_std` ):

**mean = (0.4914, 0.4822, 0.4465)  std = (0.2470, 0.2435, 0.2616)**

After normalization, a validation batch has per-channel mean ≈ 0 and std ≈ 1, which confirms the statistics. The often-copied std (0.2023, 0.1994, 0.2010) in the template is not the per-channel std of the dataset, so the computed values are used instead.

| Split | Images | Transforms (in order) |
|---|---|---|
| Train | 45,000 | `RandomCrop(32, padding=4)` → `RandomHorizontalFlip(p=0.5)` → `ToTensor()` → `Normalize(mean, std)` → `Cutout(1 hole, 16×16)` |
| Validation | 5,000 | `ToTensor()` → `Normalize(mean, std)` |
| Test | 10,000 | `ToTensor()` → `Normalize(mean, std)` |

- **RandomCrop with 4-pixel padding:** random translations of up to ±4 px, which teach the network that an object's position does not change its class.
- **RandomHorizontalFlip:** CIFAR-10 classes are left-right symmetric, so mirroring effectively doubles the variety of training images.
- **Normalize :** zero-mean, unit-variance inputs per channel keep the first layer's activations and gradients well scaled, which speeds up and stabilizes optimization.
- **Cutout (16×16, applied after Normalize):** one random square is set to 0, which is the dataset mean colour after normalization. This forces the network to use several parts of the object rather than one discriminative patch, and acts as a regularizer.
- Validation and test images are only converted and normalized, so evaluation is deterministic.

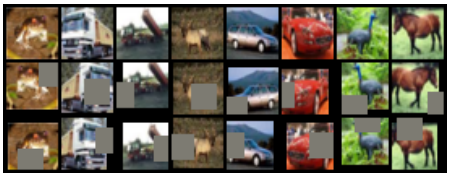


Figure 1. Row 1: original training images. Rows 2 and 3: two random augmentations of the same images (crop shift, flip and Cutout).

### (b) Training with the chosen configuration

Following the instruction to prefer the configuration with the best accuracy, the model is trained with the best configuration found in Q2 (see Q4). That is the 150-epoch run of the epochs sweep, W&B run `p78f6vr6` :

| activation | BatchNorm | optimizer | lr | momentum | weight decay | LR schedule | batch size | epochs | seed |
|---|---|---|---|---|---|---|---|---|---|
| ReLU | yes | Nesterov-SGD | 0.1 | 0.9 | 5e-4 | cosine to 0 | 128 | 150 | 42 |

Command: `python -m src.train --config configs/best.yaml`. For reference, the initial baseline (`configs/baseline.yaml`: SGD, lr 0.05, 100 epochs) reached 88.64% validation and 87.67% test accuracy.

### (c) Final test top-1 accuracy and curves

> **Final test top-1 accuracy: 88.40%** (10,000 test images), using the checkpoint with the best validation accuracy (88.80%, epoch 149). Re-evaluating the submitted weights with `python -m src.evaluate --checkpoint checkpoints/best_model.pth` gives the same numbers.

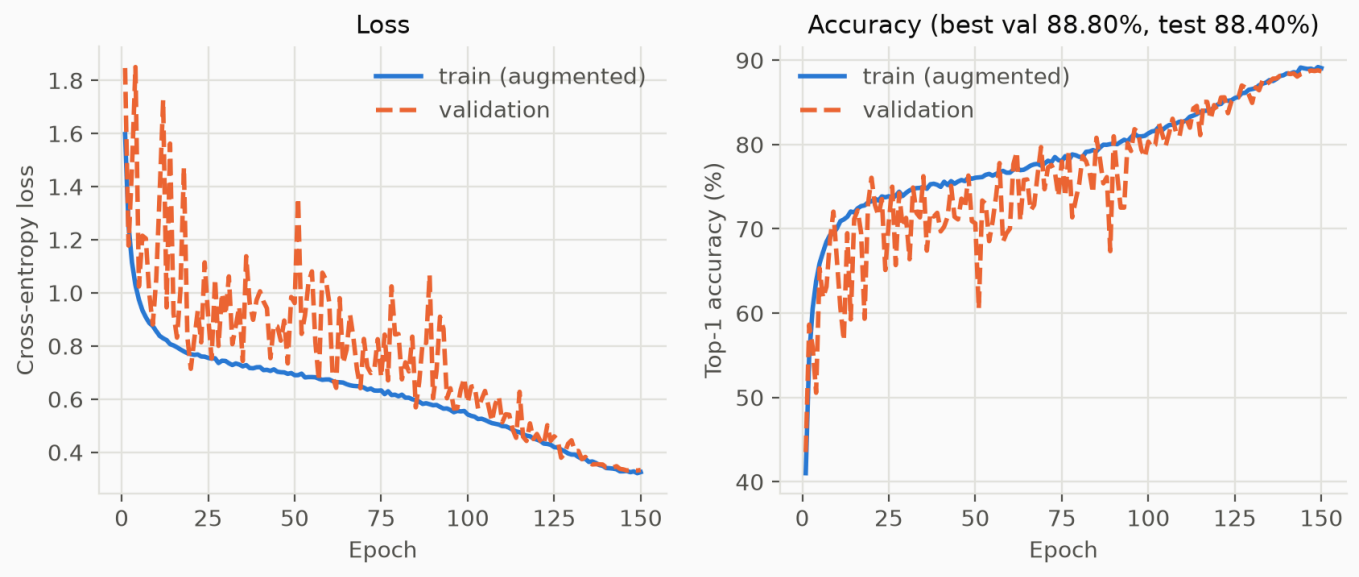



Figure 2. Loss and accuracy per epoch for the best configuration. The validation curves are noisy while the LR is high and become smooth as the cosine schedule anneals it.

Training accuracy stays at or slightly below validation accuracy for almost the whole run, because it is measured on augmented images with Cutout. The final gap is about 0.3 points (89.06% train vs 88.68% validation at the last epoch). The network is therefore not overfitting: with 262k parameters it is capacity-limited.

## Question 2. Model Performance on Different Configurations

### (a) Varying the activation function

**Experiment** (W&B sweep `y9279jnd`, 10 runs): a grid over activation ∈ {ReLU, Sigmoid, Tanh, SiLU, GELU} × BatchNorm ∈ {on, off}, with 50 epochs and all other hyperparameters at the defaults. BatchNorm is included because it largely hides the differences between activations, so turning it off shows each activation's own behaviour.

| Activation | BN on: val | BN on: test | BN off: val | BN off: test | Val @ epoch 10 (BN on / off) | First epoch ≥ 80% (BN on / off) | Final train loss (BN on) |
|---|---|---|---|---|---|---|---|
| **ReLU** | **86.94** | **86.11** | **85.76** | **84.96** | **56.2 / 71.1** | **34 / 22** | **0.417** |
| GELU | 86.92 | 85.93 | 85.94 | 84.62 | 73.4 / 73.2 | 26 / 22 | 0.417 |
| SiLU | 86.08 | 85.13 | 86.06 | 84.93 | 71.7 / 73.4 | 31 / 22 | 0.469 |
| Tanh | 77.12 | 75.99 | 76.36 | 75.20 | 45.1 / 64.0 | never / never | 0.728 |
| Sigmoid | 67.16 | 65.79 | 10.60 | 10.00 | 29.8 / 9.9 | never / never | 1.035 |

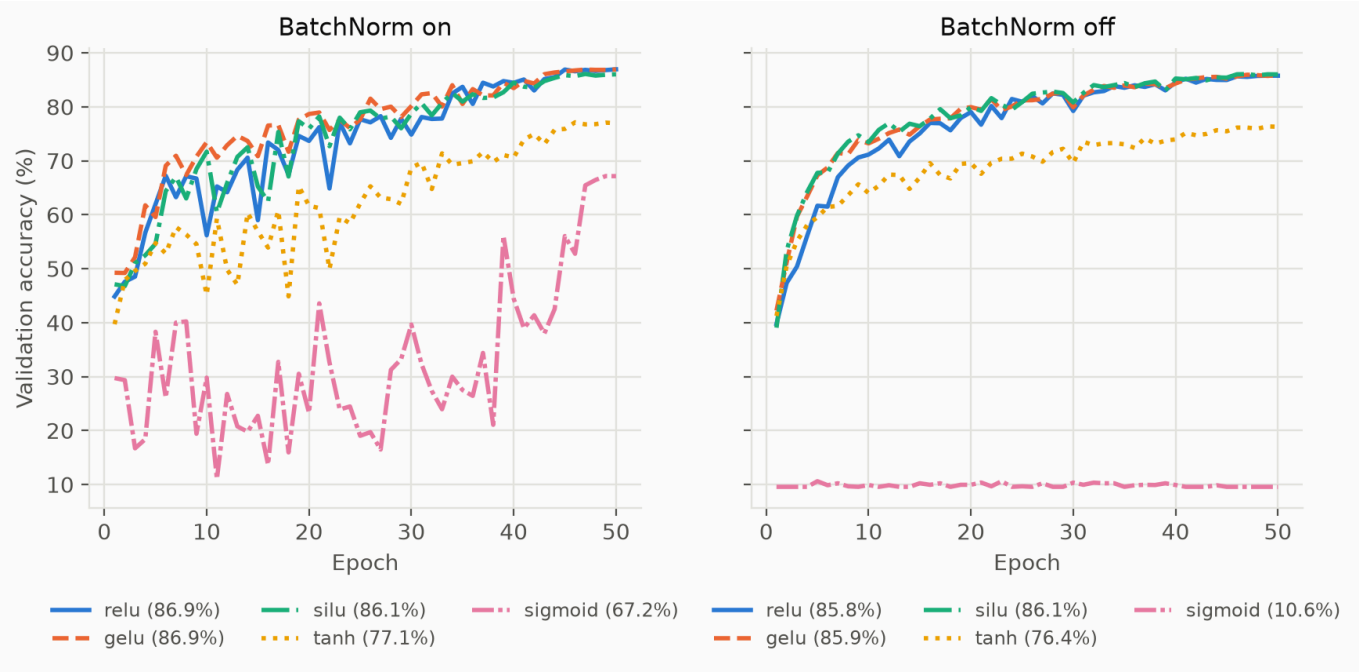



Figure 3. Validation accuracy per epoch for each activation, with BatchNorm on (left) and off (right). The legend gives each run's best validation accuracy.

- **ReLU, GELU and SiLU perform best and are effectively tied (86–87%).** All three are non-saturating for positive inputs, where the derivative is about 1, so the gradient reaches the first conv layer without shrinking. GELU and SiLU are smooth and let small negative values through, but in a network only four conv layers deep this brings no measurable benefit. The 0.02-point gap between ReLU and GELU is far below run-to-run noise. GELU did converge fastest early on (73% at epoch 10 with BN, vs 56% for ReLU).
- **Tanh is about 10 points lower (77%).** It is zero-centred and its maximum derivative is 1, so it trains. However, it saturates for $|x| > 2$, where the gradient goes to 0, and units stuck in saturation stop learning. Its training loss remains high (0.73 vs 0.42 for ReLU), so the network *underfits*, and the curve never reaches 80%.
- **Sigmoid is the worst.** Its derivative is at most 0.25, so the gradient shrinks by at least 4× per layer, and its outputs are always positive (not zero-centred), which biases the next layer's weight updates in one direction. **Without BatchNorm the network does not learn at all**: the training loss stays at exactly $\ln 10 = 2.303$ and accuracy at 10%, which is chance level. This is the vanishing-gradient problem in its clearest form. With BatchNorm, which re-centres and rescales each layer's inputs into the sigmoid's non-saturated range, it trains, but only to 67%, and very noisily.
- **Effect of BatchNorm.** BN improves the final accuracy of every activation (by 0.02 to 1.2 points for ReLU, GELU, SiLU and Tanh) and is essential for Sigmoid. At lr 0.05, however, BN runs are noisier early on: ReLU with BN is at 56% at epoch 10 vs 71% without BN, and reaches 80% at epoch 34 vs 22. With large updates, the running statistics BN uses at evaluation time lag behind the fast-changing weights. Once the cosine schedule lowers the LR, the BN runs catch up and finish ahead.

- **Initialization caveat.** The template's He initialization is derived for ReLU. For Tanh and Sigmoid it produces larger pre-activations than ideal (Xavier initialization would suit them), which increases saturation and contributes to their gap.

### (b) Varying the optimizer

**Experiment** (sweep `mc9888go`, 24 runs): optimizer ∈ {SGD, Nesterov-SGD, Adam, Adagrad, RMSprop, Nadam} × lr ∈ {0.1, 0.01, 0.001, 0.0001}, using ReLU with BN for 50 epochs. Each optimizer has a different natural LR range, so they are compared *at their own best LR*. Momentum 0.9 applies to SGD and Nesterov, the adaptive methods use PyTorch defaults ($\beta_1$ = 0.9, $\beta_2$ = 0.999 for Adam and Nadam; $\alpha$ = 0.99 for RMSprop), and every optimizer uses L2 weight decay of 5e-4.

| Optimizer | Best lr | Best val | Test | Val @ ep 5 | Val @ ep 10 | First epoch ≥ 80% | First epoch ≥ 85% | Val noise* |
|---|---|---|---|---|---|---|---|---|
| **Nesterov-SGD** | **0.1** | **87.56** | **86.57** | **64.3** | **67.0** | **33** | **44** | **4.1** |
| SGD (momentum) | 0.1 | 86.94 | 86.74 | 57.3 | 61.6 | 34 | 44 | 5.2 |
| Nadam | 0.001 | 86.20 | 85.17 | 63.1 | 70.1 | 25 | 37 | 4.2 |
| Adam | 0.001 | 85.94 | 85.47 | 58.4 | 73.3 | 25 | 39 | 3.9 |
| RMSprop | 0.001 | 85.90 | 84.93 | 64.6 | 61.3 | 28 | 44 | 6.9 |
| Adagrad | 0.1 | 84.88 | 83.91 | 59.4 | 65.3 | 26 | never | 4.9 |

*Mean absolute epoch-to-epoch change in validation accuracy over the first 25 epochs (higher means a more erratic trajectory).

| Best val accuracy at lr | SGD | Nesterov | Adam | Adagrad | RMSprop | Nadam |
|---|---|---|---|---|---|---|
| 0.1 | **86.94** | **87.56** | 45.88 | **84.88** | 27.30 | 45.40 |
| 0.01 | 82.94 | 82.92 | 79.08 | 79.02 | 77.00 | 80.48 |
| 0.001 | 67.72 | 67.90 | **85.94** | 59.92 | **85.90** | **86.20** |
| 0.0001 | 46.94 | 46.96 | 72.70 | 40.08 | 72.26 | 72.64 |

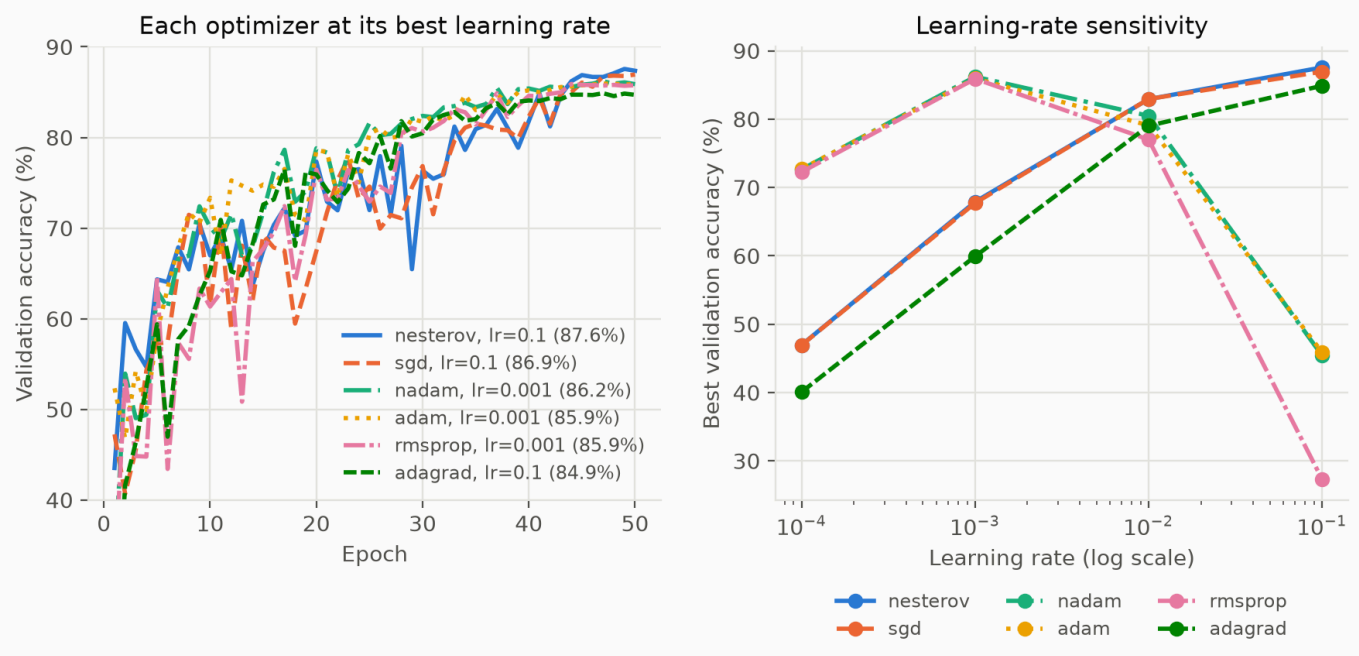



Figure 4. Left: validation accuracy of each optimizer at its best learning rate. Right: best validation accuracy as a function of learning rate (log scale).

**How each optimizer affects convergence:**

- **SGD with momentum** ($v \leftarrow \mu v + g$, $\theta \leftarrow \theta - \eta v$) moves with step size proportional to lr × gradient. The gradients of this BN network are small, so it needs a large LR. Accuracy rises steadily with LR across the whole tested range (47% → 68% → 83% → 87%). Below 0.01 it is far too slow, still underfitting after 50 epochs with a training loss above 1.0. At lr 0.1 its early trajectory is the noisiest of the SGD family, but it generalizes best: its test accuracy of 86.74% is the highest of all optimizers.
- **Nesterov-SGD** evaluates the gradient at the look-ahead point $\theta - \mu v$, so momentum is corrected *before* it overshoots. At small LRs it is indistinguishable from plain SGD (the curves coincide in Figure 4, right). At lr 0.1 it starts faster (64% vs 57% at epoch 5), its trajectory is smoother (noise 4.1 vs 5.2), and it gains +0.6 points of validation accuracy, giving the **best validation accuracy of all optimizers (87.56%)**.

- **Adam** keeps a momentum (first-moment) estimate and divides each parameter's step by the square root of a running average of $g^2$ (second moment), with bias correction. Each parameter therefore takes a step of roughly lr regardless of the gradient's scale. This makes it **converge fastest** (73% at epoch 10, 80% by epoch 25 vs 33–34 for SGD), but also makes it very LR-sensitive: it needs about 1e-3 and collapses to 46% at lr 0.1, where every parameter takes a step that is far too large. It ends 1–1.5 points below the SGD family, which reflects the usual generalization gap of adaptive methods. Part of this gap comes from the setup: with coupled L2 weight decay, the decay term is also divided by the adaptive denominator, so parameters with large gradients are barely regularized. AdamW's decoupled decay addresses this.
- **Nadam** is Adam with Nesterov momentum. The look-ahead gives the same kind of benefit as for SGD, though smaller: +0.26 points over Adam, with 85% reached at epoch 37 vs 39. Its LR sensitivity is the same as Adam's.
- **RMSprop** uses the same adaptive denominator as Adam, but has **no first-moment momentum and no bias correction**. Its first updates (when the running average of $g^2$ is still near 0) are therefore large and erratic: validation accuracy is only 31% after epoch 1, vs 52% for Adam, and it even fell from 64.6% to 61.3% between epochs 5 and 10. It has the **noisiest trajectory of all** (6.9 at its best LR, 9.5 at lr 0.01) and is the most fragile at high LR (27% at lr 0.1). At lr 1e-3 it still finishes close to Adam (85.90%).
- **Adagrad** divides by the square root of the *cumulative sum* of all past $g^2$, which only grows. Its effective learning rate therefore decays continuously throughout training. It works only with a large initial LR (0.1 gives 84.88%, while 1e-3 stalls at 60%), makes good progress early (80% by epoch 26), and then flattens out. It is the only optimizer that never reaches 85%, because by the time fine-grained progress matters its steps have become too small.

**Summary of the differences.** The SGD-type optimizers (SGD, Nesterov) are slow at first but reach the highest final and test accuracy, provided the LR is large. The adaptive optimizers (Adam, Nadam, RMSprop) reach good accuracy two to three times sooner in epochs and work at small LRs, but they are sensitive to the LR and finish slightly lower. Among them, momentum and bias correction (Adam, Nadam) make training far smoother than RMSprop. Adagrad's ever-shrinking step size makes it the weakest for long training. Nesterov-SGD at lr 0.1 was selected for Q2(c).

### (c) Varying batch size, number of epochs and learning rate

**Experiments:** sweep `qtfqhv8y` (20 runs) is a grid of batch size ∈ {32, 64, 128, 256, 512} × lr ∈ {0.01, 0.05, 0.1, 0.2} at 50 epochs. Sweep `vvd87d3w` (5 runs) varies epochs ∈ {10, 20, 50, 100, 150} at batch size 128 and lr 0.1. Both use ReLU, BN and Nesterov-SGD with a cosine schedule.

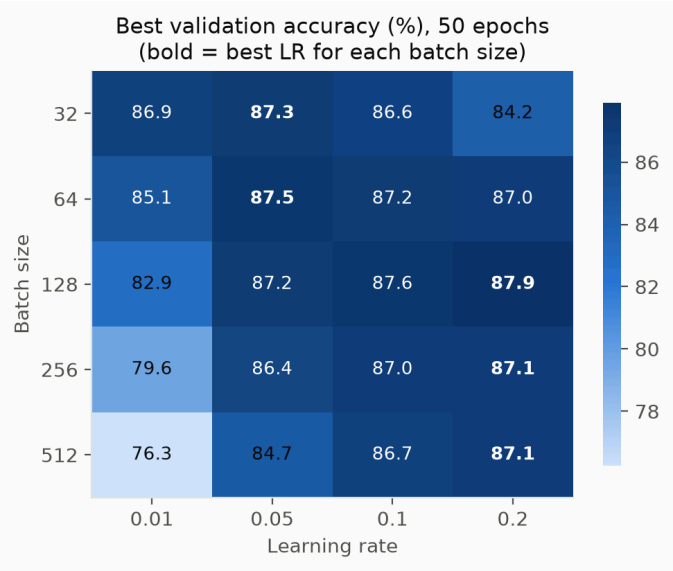



Figure 5. Best validation accuracy for each batch size × learning rate (50 epochs).

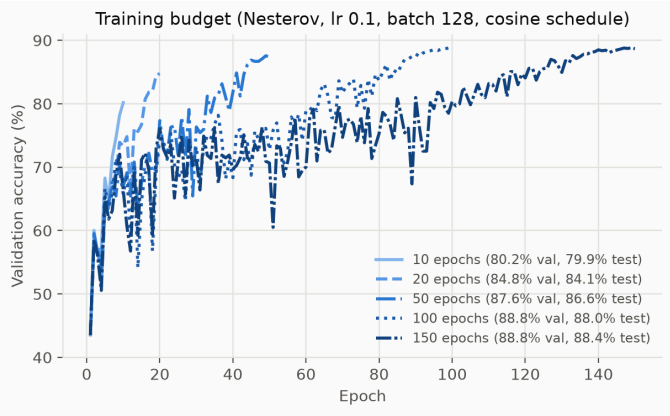



Figure 6. Validation accuracy for different epoch budgets (each run anneals the LR to 0 at its own last epoch).

| Batch size (steps/epoch) | Best lr | Best val | Val at lr 0.01 | First epoch ≥ 80% at best lr | Optimizer steps to 80% |
|---|---|---|---|---|---|
| 32 (1,407) | 0.05 | 87.28 | 86.86 | 36 | 50,652 |
| 64 (704) | 0.05 | 87.52 | 85.14 | 34 | 23,936 |
| **128 (352)** | **0.2** | **87.90** | **82.92** | **40** | **14,080** |
| 256 (176) | 0.2 | 87.12 | 79.58 | 36 | 6,336 |
| 512 (88) | 0.2 | 87.10 | 76.26 | 32 | 2,816 |

| Epochs (bs 128, lr 0.1) | 10 | 20 | 50 | 100 | 150 |
|---|---|---|---|---|---|
| Best val accuracy | 80.18 | 84.76 | 87.56 | 88.80 | **88.80** |
| Test accuracy | 79.90 | 84.06 | 86.57 | 87.98 | **88.40** |
| Final train loss | 0.681 | 0.532 | 0.408 | 0.350 | 0.325 |
| First epoch ≥ 80% val | 10 | 16 | 33 | 64 | 85 |

**Learning rate and batch size interact.**

- **Larger batches need a larger learning rate.** The best LR rises from 0.05 at batch 32–64 to 0.2 at batch 128–512. The gradient noise of SGD scales roughly with lr/B, and configurations with the same lr/B ratio perform alike: 32/0.05 gives 87.3, 64/0.1 gives 87.2 and 128/0.2 gives 87.9. This is the linear-scaling rule.
- **Small LR with a large batch underfits.** At lr 0.01, accuracy falls steadily as the batch grows (86.9 → 85.1 → 82.9 → 79.6 → 76.3). Batch 512 makes only 88 updates per epoch (4,400 in 50 epochs vs 70,350 for batch 32), so with small steps it simply does not travel far enough. Its training loss is still 0.79 at the end.
- **Large LR with a small batch is too noisy.** Batch 32 at lr 0.2 reaches only 84.2%, with the most erratic curve in the grid. Batch 64 at lr 0.2 dropped to 35% validation at epoch 10. Small batches already have noisy gradients, and a large LR amplifies the noise.
- **Convergence speed.** Measured in *optimizer steps*, large batches converge far faster: batch 512 at lr 0.1 reaches 80% after about 2,200 updates, vs about 35,000 for batch 32 at lr 0.01. Measured in *epochs* they are similar (both about 25 epochs), because each large-batch step averages more examples. In wall-clock time, large batches are faster per epoch, because fewer and larger GPU kernels use the GPU better. (Epoch times in these sweeps are not compared directly, because four agents shared the GPU.)
- **Final accuracy is not very sensitive to batch size once the LR is tuned**: the best value for every batch size lies between 87.1% and 87.9%. Batch size mainly determines training speed and the LR needed. Batch 128 with lr 0.2 was the best 50-epoch result.

**Number of epochs.** More epochs give a higher final accuracy, with clearly diminishing returns: +4.6 points from 10 to 20 epochs, +2.8 from 20 to 50, +1.2 from 50 to 100, and none from 100 to 150 in validation accuracy (+0.4 in test accuracy, with a lower training loss). Each extra epoch costs the same compute, so 150 epochs cost three times as much as 50 for +1.2 validation points. Because the cosine schedule anneals the LR to 0 at the end of each budget, every run achieves its best accuracy in its final epochs. A longer budget keeps the LR high for longer, so at any given epoch its accuracy is *lower* (the 150-epoch run only passes 80% at epoch 85, while the 10-epoch run is at 80.2% by epoch 10), but it ends at a better optimum. The gap between training and validation accuracy stays below 0.4 points even at 150 epochs, so longer training was not limited by overfitting.

## Question 3. Plots (generated with W&B)

### (a) Parallel-coordinates plot

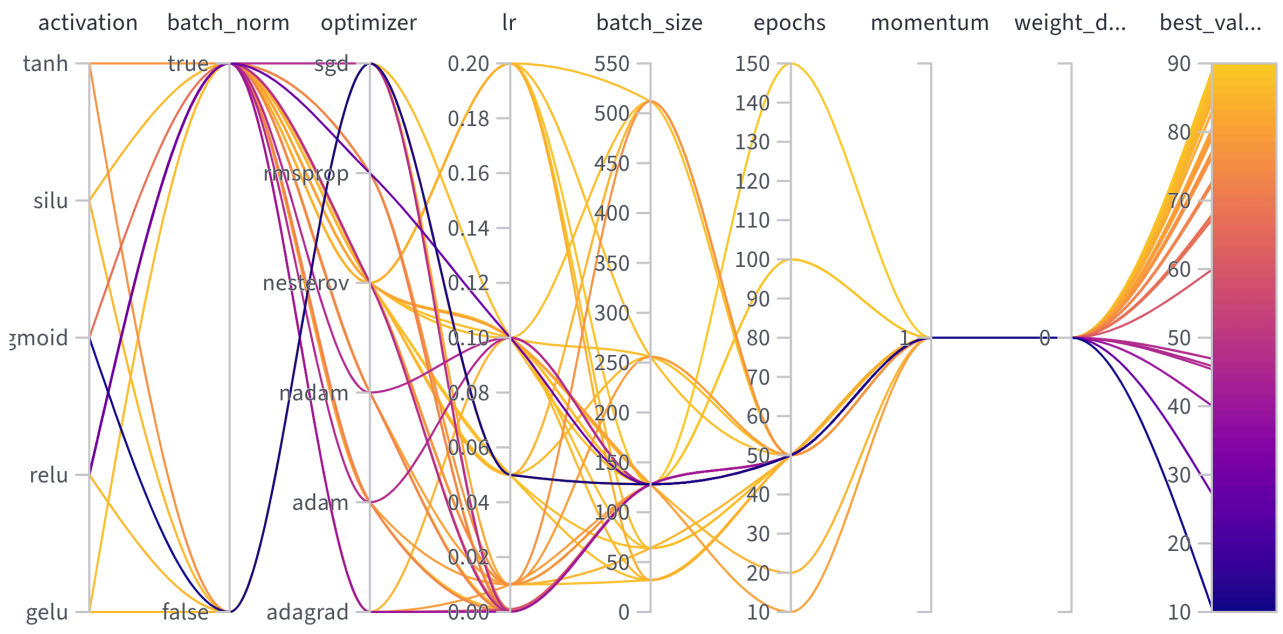



Figure 7. W&B parallel-coordinates plot of all 60 runs. Each line is a configuration (activation, BatchNorm, optimizer, lr, batch size, epochs, momentum, weight decay) coloured by its best validation accuracy. Momentum (0.9) and weight decay (5e-4) were held constant. Yellow lines (≈ 85–89%) run through ReLU, GELU or SiLU with SGD or Nesterov at lr 0.05–0.2. The dark purple lines at the bottom are Sigmoid without BN (10%) and the adaptive optimizers at lr 0.1.

### (b) Validation accuracy vs. step (scatter)

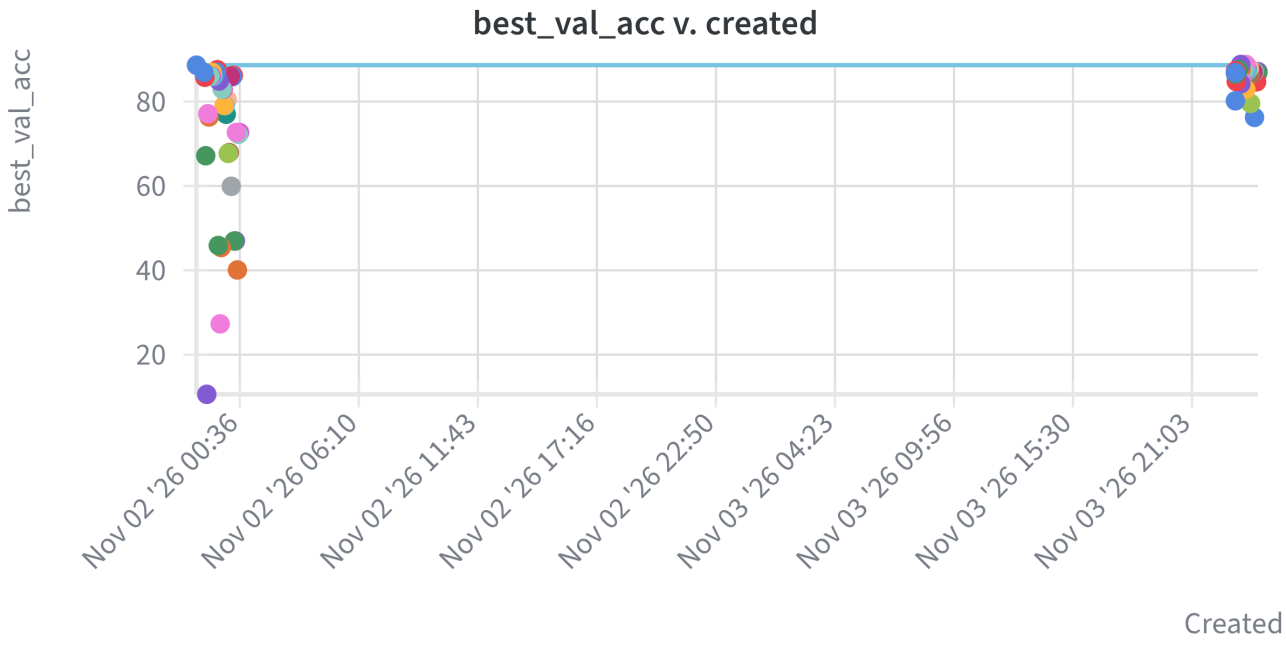



Figure 8. W&B scatter of each run's best validation accuracy against its creation time, with the running maximum (line). The left cluster is the baseline and the Q2(a)/Q2(b) sweeps, and the right cluster is the Q2(c) sweeps, run after a break. The running maximum reaches 88.80%.

### (c) Training loss, training accuracy, validation loss, validation accuracy

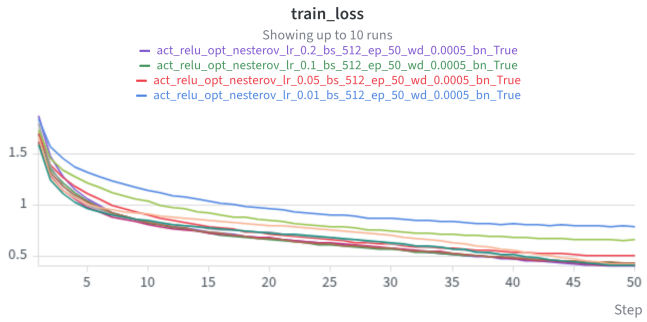



Figure 9. Training loss.

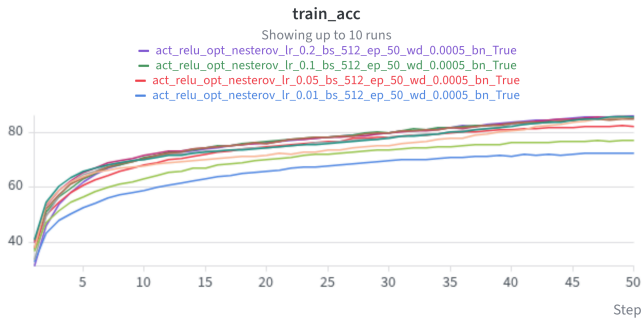



Figure 10. Training accuracy.

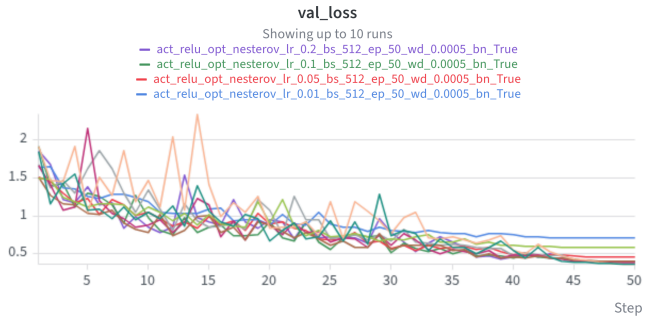



Figure 11. Validation loss.

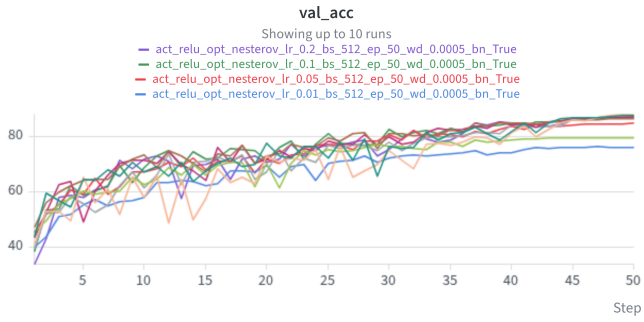



Figure 12. Validation accuracy.

Figures 9–12 are W&B line charts with step = epoch, showing up to 10 runs of the batch size × learning rate sweep. The lowest-accuracy, highest-loss curve is batch 512 at lr 0.01 (underfitting), and the runs at lr 0.1–0.2 give the highest curves.

## Question 4. Final Model Performance

The configuration with the **best validation accuracy** in the W&B parallel-coordinates plot is run `p78f6vr6` (sweep `q2c_epochs`):

| activation | batch_norm | optimizer | lr | momentum | weight_decay | scheduler | batch_size | epochs | augmentation | seed |
|---|---|---|---|---|---|---|---|---|---|---|
| relu | true | nesterov | 0.1 | 0.9 | 5e-4 | cosine | 128 | 150 | crop + flip + cutout | 42 |

> **Best validation top-1 accuracy: 88.80%** (epoch 149). Test top-1 accuracy of the same checkpoint: **88.40%**.

```
python -m src.train    --config configs/best.yaml                # re-train this configuration
python -m src.evaluate --checkpoint checkpoints/best_model.pth   # evaluate the submitted weights
```

**Reproducibility verified.** The configuration was retrained from scratch with the command above. It reproduced the original run exactly: best validation 88.80% at epoch 149, test 88.40%, and identical final-epoch metrics (train loss 0.3254, train acc 89.06%, val loss 0.3333, val acc 88.68%). In addition, the 50-epoch version of this configuration appears in three separate sweeps, and all three runs, trained while other jobs shared the GPU, gave exactly 87.56% validation and 86.57% test accuracy. (The 100-epoch run of the same configuration tied on validation accuracy at 88.80%. The 150-epoch run is the one reported because its test accuracy is higher.)

## Question 5. Reproducibility and Repository

**GitHub repository:** [https://github.com/cs25m503-sudo/vgg6-cifar10](https://github.com/cs25m503-sudo/vgg6-cifar10)

### (a) Code structure

| File | Responsibility |
|---|---|
| `src/data.py` | Normalization statistics, transforms, Cutout, seeded 45k/5k/10k split, DataLoaders |
| `src/model.py` | VGG6 (template architecture) with selectable activation and optional BatchNorm |
| `src/utils.py` | Seeding, optimizer factory (6 optimizers), LR scheduler, checkpoint save/load |
| `src/train.py` | Training loop, W&B logging, best-checkpoint saving; standalone or as a W&B sweep agent |
| `src/evaluate.py` | Evaluation (loss, top-1 accuracy), and a CLI that tests any saved checkpoint |
| `src/plot_results.py` | Regenerates the report figures for Q1 and Q2 from the W&B runs |
| `configs/*.yaml` , `sweeps/*.yaml` | Baseline and best configurations, and the four W&B sweep definitions |

Training and evaluation are kept separate: `train.py` imports `evaluate()` from `evaluate.py`, and `evaluate.py` runs independently on a saved checkpoint without W&B.

### (b) README, environment and seeds

The repository README lists the exact setup and run commands for every question, the environment (Ubuntu with Linux 7.0, Python 3.12.3, torch 2.14.1+cu126, torchvision 0.29.1, wandb 0.31.0, CUDA 12.6, RTX 4090 Laptop GPU) and all dependency versions ( `requirements.txt` ). The seed configuration is: seed 42 for Python, NumPy and PyTorch (CPU and CUDA); `cudnn.deterministic = True` and `cudnn.benchmark = False` ; a seeded validation split; a seeded DataLoader shuffle; and seeded DataLoader workers.

### (c) Trained model

The trained weights of the best configuration are in the repository at `checkpoints/best_model.pth` . The checkpoint contains the configuration needed to rebuild the model, and `python -m src.evaluate` reproduces 88.80% validation and 88.40% test accuracy from it.